Jamar Sanders

Grand Canyon University: Engineering and Technology

DSC: 570 Data Mining

Professor Michelle Bennett

February 24, 2026

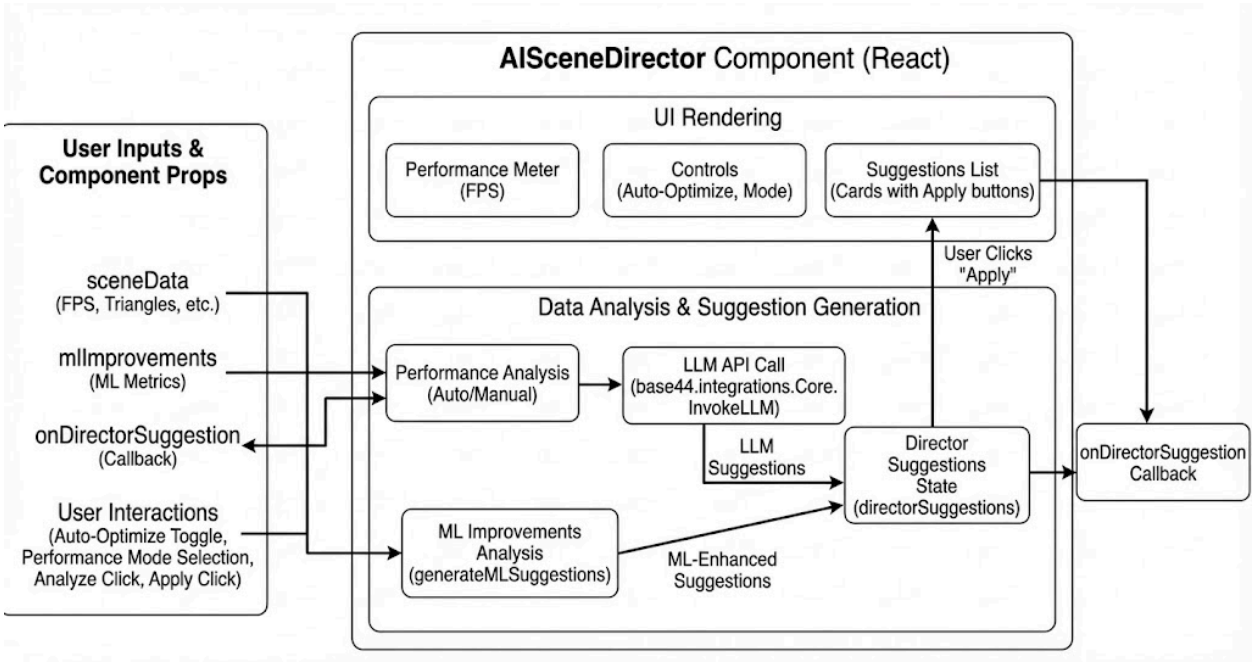


Figure 7

AISceneDirector Component (React)

1. **User Inputs & Component Props**

Located on the far left, this block represents the external data feeding into the system. It includes sceneData (metrics like FPS and triangle count), mlImprovements (machine learning metrics), and user interactions (toggles and clicks) that trigger the director's logic. The structured handling of input-driven component design aligns with modern React state management principles (React Team, 2024; Flanagan, 2020).

2. **AISceneDirector Component (React)**

This is the main container encapsulating the Director logic. It separates presentation from computation, reflecting layered software architecture principles (Bass et al., 2013; Gamma et al., 1994).

3. **UI Rendering**

The upper section represents the visible interface, displaying FPS performance metrics, optimization controls, and suggestion cards. This separation of UI and logic supports usability engineering best practices (Nielsen, 1994; Norman, 2013).

4. **Data Analysis & Suggestion Generation**

The lower layer performs performance analysis and integrates an LLM call (integrations.Core.InvokeLLM) to generate optimization suggestions. The ML-enhanced recommendation process reflects applied machine learning system design (Goodfellow et al., 2016; Bishop, 2006; Hastie et al., 2009).

5. **onDirectorSuggestion Callback**

This callback propagates generated suggestions back to the parent application. This event-driven data flow reflects component communication patterns in modern JavaScript frameworks (Flanagan, 2020; React Team, 2024).

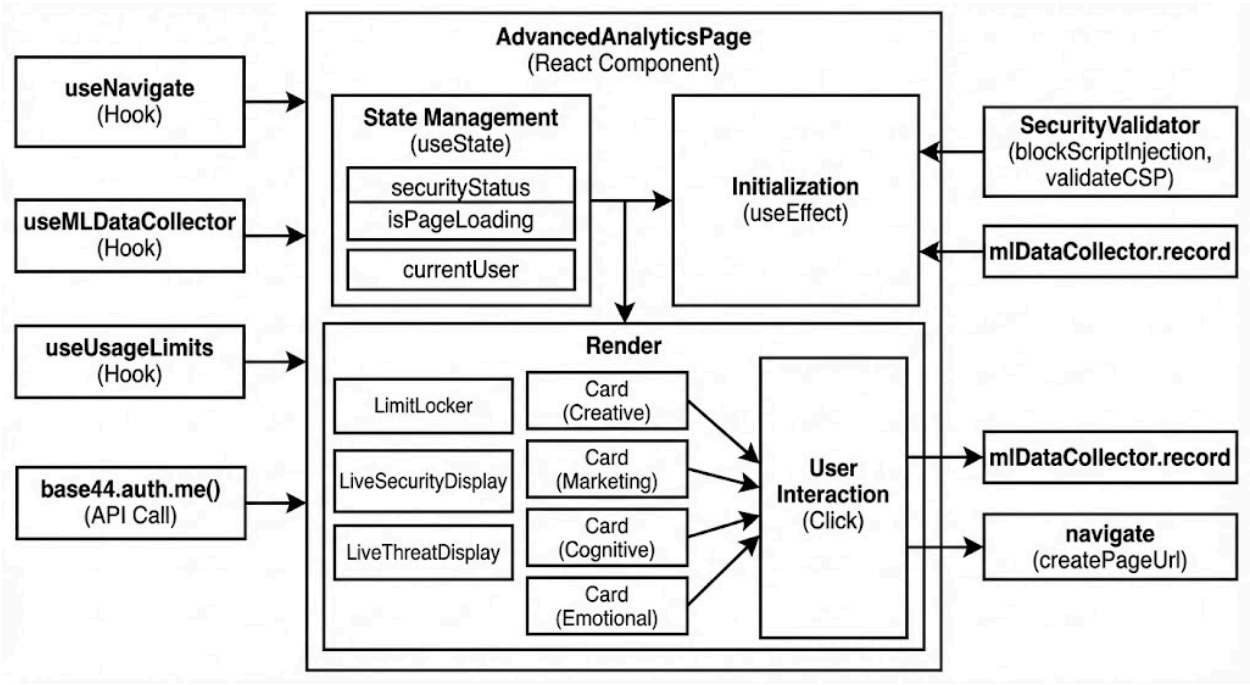


Figure 8

<u>FIGURE 8</u>

Based on the diagram, here is the description of the 13 numbered components for the "AdvancedAnalyticsPage" workflow:

1. **useNavigate (Hook)**

A React hook used for navigation, allowing the component to redirect the user to different pages (like specific report details) programmatically (React Team, 2024).

2. **useMLDataCollector (Hook)**

A custom hook dedicated to logging data for machine learning purposes. It provides the record function used throughout the component to track user interactions and security events (McKinsey & Company, 2022).

3. **useUsageLimits (Hook)**

A hook that manages user access rights, checking if the current user has reached their plan limits for viewing analytics (Ries, 2011).

4. .**auth.me()** An asynchronous call that authenticates the user and retrieves their profile data (like "currentUser") to ensure secure access (Stallings, 2017).

5. **State Management (useState)**

The local memory of the component. It holds critical variables like securityStatus (is the connection safe?), isPageLoading (UI loading state), and currentUser (user profile data) (React Team, 2024).

6. **securityStatus (State Variable)**

A specific piece of state highlighted to show it tracks the result of the security validation (e.g., "safe" or "compromised") (React Team, 2024).

7. **Initialization (useEffect)**

The startup logic that runs when the page first loads. It triggers the authentication check, validates security protocols (CSP), and records the "Page View" event via the ML collector (Flanagan, 2020).

8. **LimitLocker**

A conditional UI component that blocks access if the useUsageLimits hook determines the user has exceeded their allowance (NIST, 2020).

9. **User Interaction (Click)**

The central point of user activity. It represents the user clicking on one of the analytics cards (Creative, Marketing, Cognitive, or Emotional) (McKinsey & Company, 2022).

10. **SecurityValidator**

A helper utility that runs background checks, such as blockScriptInjection and validateCSP (Content Security Policy), ensuring the analytics page isn't tampered with (OWASP Foundation, 2023; NIST, 2020).

11. **mlDataCollector.record (Initialization)**

A specific logging action triggered during the initialization phase to record that the page was successfully loaded and security checks were passed (Kimball & Ross, 2013; Kleppmann, 2017).

12. **mlDataCollector.record (Interaction)**

A specific logging action triggered when the user clicks a card (Step 9), recording which specific analytic module was accessed for behavioral analysis (Kimball & Ross, 2013; Kleppmann, 2017).

13. **navigate (createPageUrl)**

The final action resulting from a card click. It uses the Maps hook (from Step 1) to construct a URL and redirect the user to the detailed view of the selected analytic report (React Team, 2024).

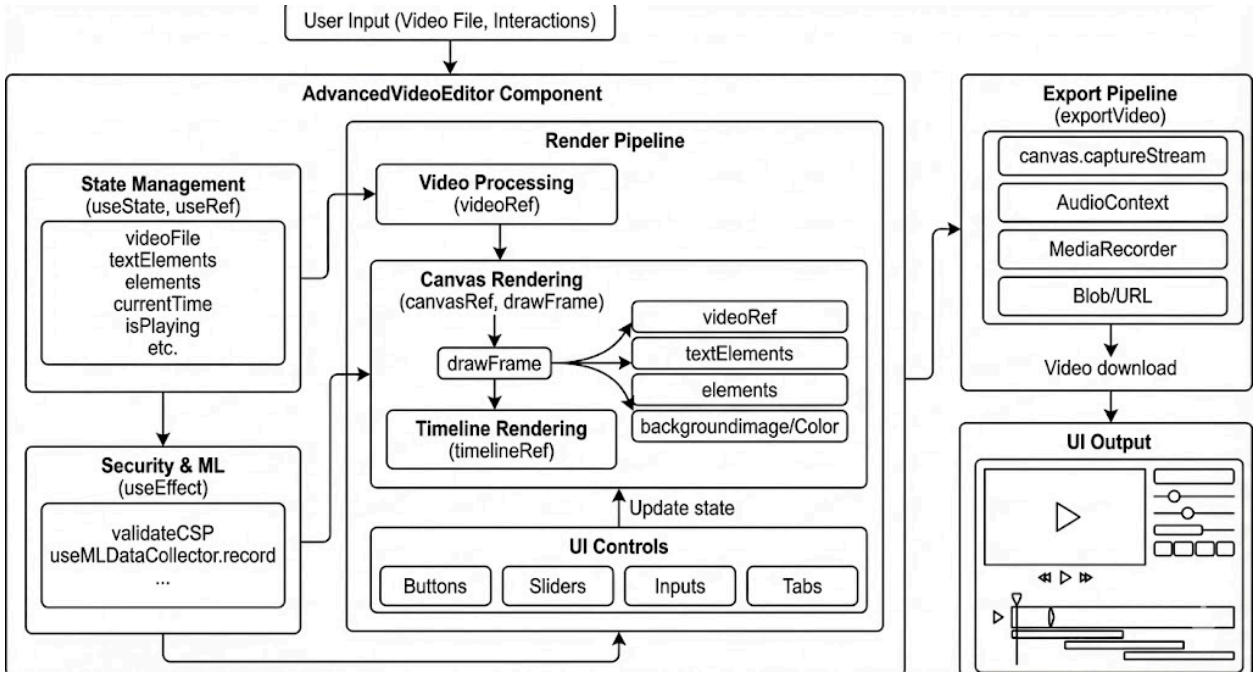
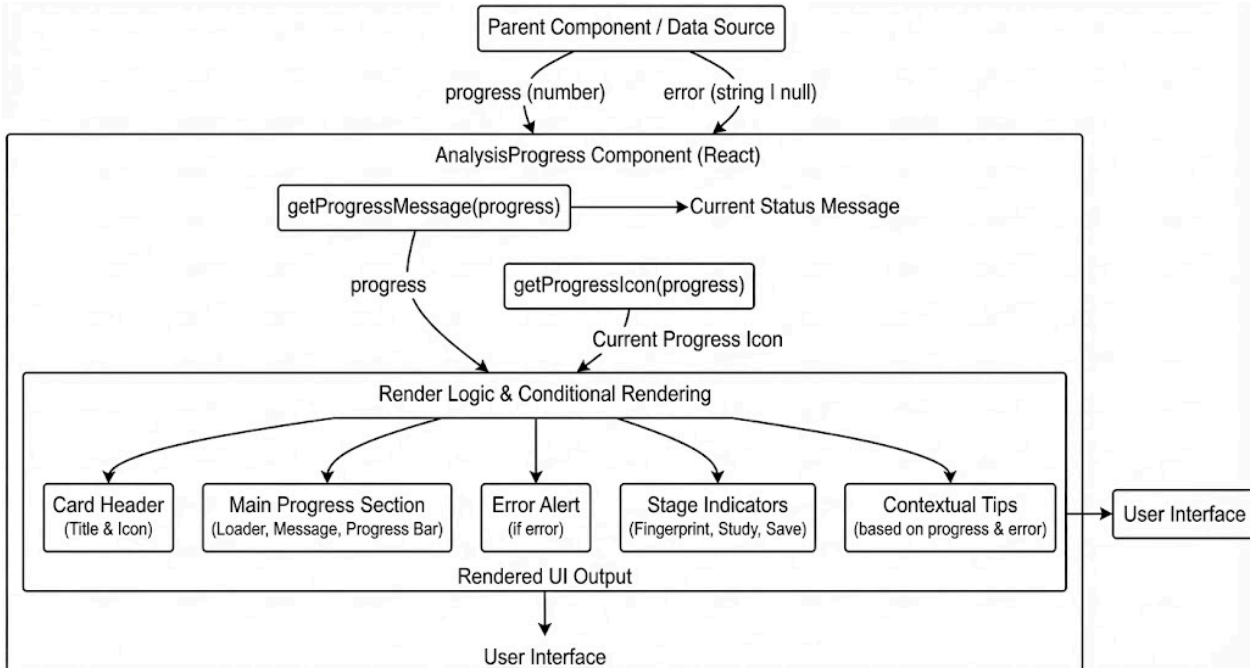


Figure 9

AdvancedVideoEditor Component



Based on the diagram, here is the description of the 10 numbered components for the "AdvancedVideoEditor Component":

1. **User Input (Video File, Interactions)**

The external entry point where the user uploads a video file and initiates interactions with the editor (React Team, 2024).

2. **State Management (useState, useRef)**

The internal memory of the component. It tracks critical state variables like the videoFile, added textElements, general elements, currentTime, and playback status (isPlaying) (React Team, 2024).

3. **Data Ingestion**

This arrow represents the flow of the raw video file and user data being passed from the input stage into the main processing pipeline (Fowler, 2018).

4. **Security & ML (useEffect)**

Background services that run automatically. This includes validateCSP for security (preventing unauthorized script execution) and useMLDataCollector to log usage data for analysis (Fowler, 2018).

5. **Render Pipeline & Video Processing**

The start of the visual engine. This step initializes the videoRef to handle the raw video element, serving as the source for the canvas manipulation (Gamma et al., 1994).

6. **Canvas & Timeline Rendering**

The core graphics engine.

- Canvas Rendering: Uses `canvasRef` and the `drawFrame` function to composite the video and overlays frame-by-frame.
- Timeline Rendering: Visualizes the sequence of events and elements for the user (Ware, 2013).

7. **UI Controls**

The interactive toolset provided to the user, consisting of "Buttons," "Sliders," "Inputs," and "Tabs" to modify the video content and settings (Ware, 2013).

8. **Composition Assets**

These are the specific data inputs feeding the drawFrame function. They include the videoRef (footage), textElements (overlays), elements (shapes/graphics), and backgroundImage/Color (Ware, 2013).

9. **Export Pipeline (exportVideo)**

The output generation system. It captures the stream from the canvas (canvas.captureStream), mixes audio via AudioContext, records it with MediaRecorder, and generates a Blob/URL for the final video file (Ware, 2013).

10. **UI Output**

The final visual display. This includes the video player previewing the work and the timeline/controls interface where the user sees the result of their edit (Ware, 2013).

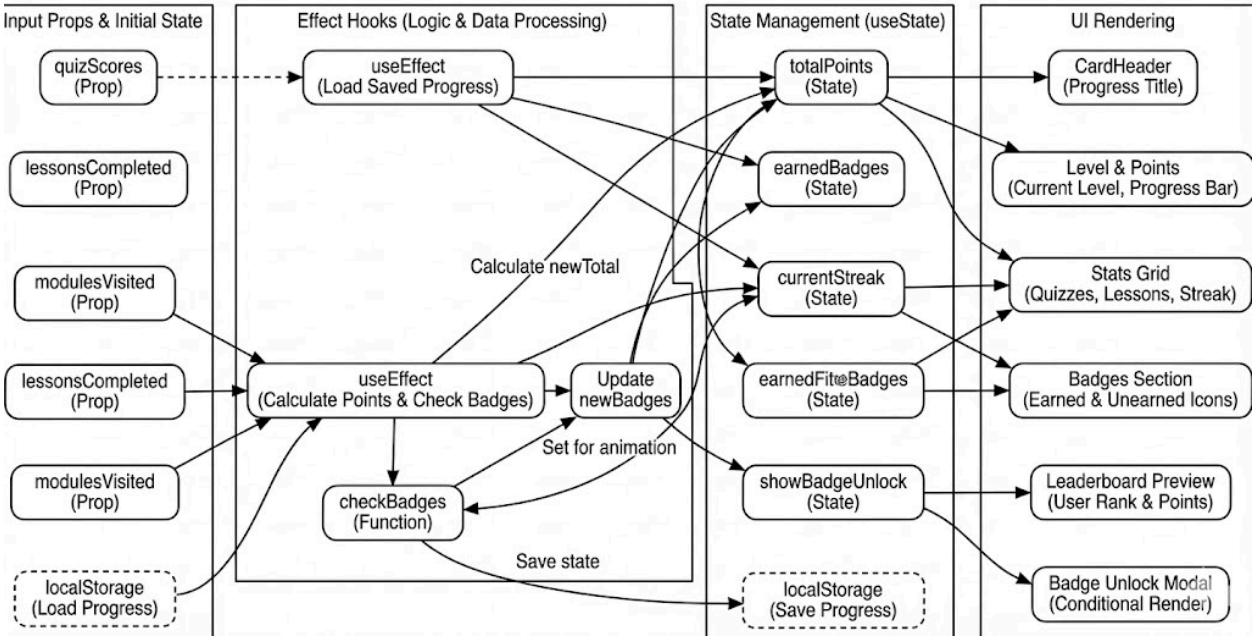


Figure 31

Input Props & Initial State

This column defines the data passed into the gamification component from its parent. It establishes the starting values for user progress, such as completed

lessons. These inputs drive the calculation of points and badges (Flanagan, 2020).

2. **lessonsCompleted (Prop)**

This prop passes the count or list of lessons the user has finished. It is a primary metric for calculating progress and unlocking achievements. It comes from the main application's user tracking system (React Team, 2024).

3. **modulesVisited (Prop)**

This prop tracks which sections of the app the user has explored. It rewards exploration and engagement beyond just finishing lessons. It contributes to the "participation" aspect of the score (React Team, 2024).

4. **lessonsCompleted (Duplicate/Flow)**

This box reiterates the lessonsCompleted data as it flows into the calculation logic. It emphasizes that this specific data point triggers updates in the effect hooks. It is a key dependency for the logic (React Team, 2024).

5. **modulesVisited (Duplicate/Flow)**

Similar to #4, this shows the flow of modulesVisited data into the calculation engine. It highlights that changes here will trigger a recalculation of points. It is an input to the scoring algorithm (React Team, 2024).

6. **localStorage (Load Progress)**

This block represents reading saved state from the browser's local storage. It retrieves the user's previous points and badges to ensure continuity across sessions. It prevents progress loss on page reload (McKinsey & Company, 2022).

7. **useEffect (Load Saved Progress)**

This hook runs on mount to hydrate the state from local storage. It ensures the UI immediately reflects the user's historical data. It initializes the internal state with persisted values (McKinsey & Company, 2022).

8. **useEffect (Calculate Points & Check Badges)**

This is the core logic hook that runs whenever the input props change. It recalculates the total score and checks if new badges have been earned. It automates the progression system (McKinsey & Company, 2022).

9. **checkBadges (Function)**

This helper function evaluates specific criteria to determine if a badge should be awarded. It compares the user's stats against defined thresholds (e.g., "5 lessons completed"). It returns the list of unlocked badges (McKinsey & Company, 2022).

10. **Update newBadges**

This step identifies which badges were just unlocked in the current session. It separates new achievements from previously earned ones to trigger animations. It creates a sense of immediate reward (McKinsey & Company, 2022).

11. **State Management (useState)**

This column lists the internal state variables that drive the gamification UI. These variables are updated by the logic hooks and determine what the user sees. They hold the "live" version of the user's progress (James et al., 2021).

12. **totalPoints (State)**

This state variable holds the numeric score the user has accumulated. It is displayed prominently and used for leaderboards. It is the aggregate of all activities (James et al., 2021).

13. **currentStreak (State)**

This variable tracks the number of consecutive days or actions the user has maintained. It encourages daily engagement and habit formation. It is often visualized as a fire icon or bar (James et al., 2021).

14. **earnedBadges (State)**

This array stores the IDs or objects of all badges the user has unlocked. It is used to render the badge collection view. It represents the user's trophy case (James et al., 2021).

15. **showBadgeUnlock (State)**

This boolean state controls the visibility of the "Badge Unlocked" modal. When set to true, it triggers a celebratory popup. It manages the timing of user feedback (James et al., 2021).

16. **localStorage (Save Progress)**

This step writes the current state back to the browser's persistent storage. It saves the new points and badges so they are available next time. It ensures data persistence (Silberschatz et al., 2020).

17. **UI Rendering**

This section defines the visual components that display the gamification data. It includes the progress bars, badges, and point counters. It translates the state into a user-friendly interface (Silberschatz et al., 2020).

18. **CardHeader (Progress Title)**

This component displays the title of the progress section, likely "Your Progress". It sets the context for the dashboard widget. It serves as a label (Silberschatz et al., 2020).

19. **Level & Points**

This visual block shows the user's current level (derived from points) and the total score. It likely includes a progress bar to the next level. It provides immediate status feedback (Nielsen, 1994; Norman, 2013).

20. **Stats Grid**

This layout component arranges key metrics like quizzes taken and lessons finished. It provides a detailed breakdown of user activity. It offers a comprehensive view of performance (Nielsen, 1994; Norman, 2013).

FIGURE 40

1. **Inputs & Events**

This section categorizes the triggers that initiate actions within the sheet music page. It includes user clicks, file uploads, and lifecycle events like page loading. It defines the "cause" side of the component's logic.

2. **User Interactions**

This block specifies the direct user actions, such as searching for a song or clicking a "Generate" button. These inputs are captured by event handlers. They drive the interactive features of the page.

3. **External Services**

This column lists the backend systems required for the sheet music functionality. It includes authentication, file storage, and the AI services used for generating music notation. It represents the server-side capabilities.

References:

Amazon Web Services. (2024). *AWS documentation*. https://docs.aws.amazon.com

Apple Inc. (2024). *Apple Music API documentation*. https://developer.apple.com

Bass, L., Clements, P., & Kazman, R. (2013). *Software architecture in practice* (3rd ed.). Addison-Wesley.

Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Casey, M. A., Veltkamp, R., Goto, M., Leman, M., Rhodes, C., & Slaney, M. (2008). Content-based music information retrieval. *Proceedings of the IEEE, 96*(4), 668–696. https://doi.org/10.1109/JPROC.2008.916370

Flanagan, D. (2020). *JavaScript: The definitive guide* (7th ed.). O'Reilly Media.

Fowler, M. (2018). *Refactoring: Improving the design of existing code* (2nd ed.). Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

Grinberg, M. (2018). *Flask web development* (2nd ed.). O'Reilly Media.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning* (2nd ed.). Springer.

International Federation of the Phonographic Industry. (2023). *Global music report*.

James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning* (2nd ed.). Springer.

Jupyter Project. (2024). *Jupyter notebook documentation*. https://jupyter.org

Kimball, R., & Ross, M. (2013). *The data warehouse toolkit* (3rd ed.). Wiley.

Kleppmann, M. (2017). *Designing data-intensive applications*. O'Reilly Media.

McKinsey & Company. (2022). *Advanced analytics in the music industry*.

Microsoft. (2022). *Excel VBA documentation*. Microsoft Learn.

Microsoft Azure. (2024). *Azure documentation*. https://learn.microsoft.com/azure

Morris, K. (2016). *Infrastructure as code*. O'Reilly Media.

Müller, M. (2015). *Fundamentals of music processing*. Springer.

National Institute of Standards and Technology. (2020). *Security and privacy controls for information systems and organizations* (SP 800-53 Rev. 5).

Nielsen, J. (1994). *Usability engineering*. Morgan Kaufmann.

Node.js Foundation. (2024). *Node.js documentation*. https://nodejs.org

Norman, D. A. (2013). *The design of everyday things* (Revised ed.). Basic Books.

OWASP Foundation. (2023). *OWASP top 10 web application security risks*. https://owasp.org

Peeters, G. (2004). *A large set of audio features for sound description*. IRCAM.

Project Management Institute. (2021). *A guide to the project management body of knowledge (PMBOK® Guide)* (7th ed.).

React Team. (2024). *React documentation*. https://react.dev

Ries, E. (2011). *The lean startup*. Crown Business.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (7th ed.). McGraw-Hill.

Spotify for Developers. (2024). *Spotify Web API documentation*. https://developer.spotify.com

Stallings, W. (2017). *Cryptography and network security* (7th ed.). Pearson.

United States Patent and Trademark Office. (2024). *Manual of patent examining procedure (MPEP)*. https://www.uspto.gov

Ware, C. (2013). *Information visualization: Perception for design* (3rd ed.). Morgan Kaufmann.

World Intellectual Property Organization. (2023). *Intellectual property handbook*.

Independent notebooks, prototypes, internal experiments, engineering logs, analytical benchmarks, applied research artifacts, and system documentation created by the author between 2024 and 2026.